FILTERS IN ADVANCED JAVA

1. What is a Filter?

A **Filter** is a Java component that intercepts an HTTP request, an HTTP response, or both before or after they reach a Servlet.

Filters are commonly used for:

- Authentication and authorization
- Logging requests
- Measuring request-processing time
- Setting request or response encoding
- Adding security headers
- Blocking unwanted requests

Example: Before a user opens the dashboard, a Filter checks whether the user is logged in.

---

2. How Does a Filter Work?

The basic request flow is:

Browser → Filter → Servlet → Filter → Browser

1. The browser sends a request.
2. The Filter executes its pre-processing logic.
3. `chain.doFilter()` passes control to the next Filter or Servlet.
4. The Servlet processes the request.
5. Control returns to the Filter after `chain.doFilter()`.
6. The response goes back to the browser.

A Filter can also stop a request by not calling `chain.doFilter()`.

---

3. Important Methods of the Filter Interface

The `jakarta.servlet.Filter` interface provides three lifecycle methods.

Method	Purpose
`init()`	Called when the Filter is initialized
`doFilter()`	Executes the filtering logic for matching requests
`destroy()`	Called when the Filter is taken out of service

Method signatures

```
public void init(FilterConfig config)
        throws ServletException;

public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException;

public void destroy();
```

`doFilter()` is the main method used to process requests.

---

4. How to Create a Filter

There are three basic steps:

1. Create a Java class.
2. Implement the `Filter` interface.
3. Register it using `@WebFilter` or `web.xml`.

Example: MyFilter.java

```
import java.io.IOException;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.annotation.WebFilter;

@WebFilter("/home")
public class MyFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain)
            throws IOException, ServletException {

        System.out.println("Before Servlet");

        chain.doFilter(request, response);

        System.out.println("After Servlet");
```

```
    }
}
```

What does this code do?

- `@WebFilter("/home")` maps the Filter to requests matching `/home`.
- The first message is printed before the request reaches the Servlet.
- `chain.doFilter()` allows processing to continue.
- The second message is printed after downstream processing returns.

The Servlet code does not need to change just to add this Filter.

---

5. What Is FilterChain?

`FilterChain` allows the request to continue to the next Filter or the target resource.

```
chain.doFilter(request, response);
```

If you call `chain.doFilter()`:

- The request continues to the next Filter or Servlet.
- The response can return through the Filter chain.

If you do not call it:

- The request does not continue through the remaining chain.
- You can block access or send a response directly.

Example: Blocking a request

```
if (request.getParameter("blocked") != null) {
    response.getWriter().println("Request Blocked");
    return;
}

chain.doFilter(request, response);
```

This is only a demonstration. Real access control should be based on properly validated authentication and authorization rules, not simply the presence of a query parameter.

---

6. Filter Lifecycle

The Filter lifecycle is managed by the Servlet container.

`init()`

Called when the Filter is initialized. It is suitable for reading configuration or preparing resources.

`doFilter()`

Called for each matching request that passes through the Filter.

`destroy()`

Called when the Filter is being taken out of service, such as during application shutdown or redeployment.

Example:

```
@Override
public void init(FilterConfig config) {
    System.out.println("Filter Initialized");
}

@Override
public void destroy() {
    System.out.println("Filter Destroyed");
}
```

These methods are optional to override when your implementation does not need them.

---

7. Real-World Example: Authentication Filter

Suppose a user tries to access a dashboard without logging in.

The Filter checks whether a session exists and whether it contains a login attribute.

```
import java.io.IOException;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import jakarta.servlet.annotation.WebFilter;

@WebFilter("/dashboard")
public class AuthenticationFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
```

```
            FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req =
                (HttpServletRequest) request;

        HttpServletResponse res =
                (HttpServletResponse) response;

        HttpSession session = req.getSession(false);

        if (session != null
                && session.getAttribute("user") != null) {

            chain.doFilter(request, response);

        } else {
            res.sendRedirect(
                req.getContextPath() + "/login"
            );
        }
    }
}
```

How does it work?

- If a session exists and contains the `user` attribute, the request continues.
- Otherwise, the user is redirected to `/login`.
- `getSession(false)` does not create a new session merely to check authentication.

Important: Your login Servlet must set the `user` session attribute after successful authentication. In production, ensure that authorization also checks the user's required permissions.

---

8. URL Patterns in `@WebFilter`

Pattern	Meaning
`@WebFilter("/home")`	Matches `/home`
`@WebFilter("/dashboard")`	Matches `/dashboard`
`@WebFilter("/*")`	Matches all URL paths in the application
`@WebFilter("*.jsp")`	Matches JSP paths
`@WebFilter("/admin/*")`	Matches paths under `/admin/`

For example:

```
@WebFilter("/admin/*")
```

can be used to protect administrative pages, provided the Filter performs the necessary authentication and authorization checks.

---

9. Multiple Filters

An application can have multiple Filters.

For example:

Browser → LoggingFilter → AuthenticationFilter → Servlet

- `LoggingFilter` records request information.
- `AuthenticationFilter` checks login status.
- The Servlet handles the actual request.

When multiple Filters match a request, their order depends on their registration and mapping configuration. Do not assume that separate `@WebFilter` annotations guarantee a particular order.

---

10. Filter vs Listener vs Servlet

Feature	Filter	Listener	Servlet
Main purpose	Intercept requests and responses	Observe lifecycle and attribute events	Handle requests
Main method	`doFilter()`	Depends on listener interface	`doGet()`, `doPost()`, etc.
Automatically invoked?	Yes, for matching requests	Yes, when its event occurs	Yes, when mapped request arrives
Can block a request?	Yes	Not normally	Yes, by its response logic
Example	Authentication	Session tracking	Login page processing

---

11. Registering a Filter Using `web.xml`

Instead of `@WebFilter`, you can configure the Filter in `WEB-INF/web.xml`.

```
<filter>
    <filter-name>MyFilter</filter-name>
    <filter-class>com.demo.MyFilter</filter-class>
</filter>

<filter-mapping>
    <filter-name>MyFilter</filter-name>
    <url-pattern>/home</url-pattern>
</filter-mapping>
```

Use either annotation-based registration or XML configuration as appropriate for your project. Avoid unintentionally registering the same Filter twice.

---

12. Important Points to Remember

1. A Filter implements the `Filter` interface.
2. `@WebFilter` registers and maps the Filter.
3. `doFilter()` contains the main filtering logic.
4. `chain.doFilter()` continues the request.
5. A Filter can execute logic before and after downstream processing.
6. A Filter can block a request by not continuing the chain.
7. Multiple Filters can be used in one application.
8. Filters do not replace Servlets; they work alongside them.
9. `init()` and `destroy()` are lifecycle methods, not methods called for every request.
10. For Tomcat 10+, use `jakarta.servlet` imports. Tomcat 9 uses `javax.servlet`.

---

13. Practice Questions

1. What is a Filter in Advanced Java?
2. Explain the Filter lifecycle.
3. What is the purpose of `chain.doFilter()`?
4. What happens if `chain.doFilter()` is not called?
5. Differentiate between Filter, Listener, and Servlet.
6. Write a Filter that measures request-processing time.
7. Write an Authentication Filter for a dashboard.
8. Explain `@WebFilter` and URL patterns.
9. Can multiple Filters process the same request?
10. Explain two real-world applications of Filters.